Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 8: Template Inheritance & Modular UI (Flask)

Task 8.1 — Programmatic active navigation highlighting

Aim. Mark the current nav link server-side via an active_page context variable.

Theory. Each route passes its identifier; the base template compares and injects the .active class — no client JS needed.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}Lab Sheet 08{% endblock %} | MySite — Saiyed Shizain</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='css/style.css') }}">
</head>
<body>
  <!-- TASK 8.1: Master navigation bar. The active page name is passed from
       each route as `active_page`; a match injects the 'active' class. -->
  <nav class="navbar">
    <div class="brand">MySite</div>
    <ul class="nav-links">
      <li>
        <a href="{{ url_for('home') }}"
           class="{{ 'active' if active_page == 'home' else '' }}">Home</a>
      </li>
      <li>
        <a href="{{ url_for('about') }}"
           class="{{ 'active' if active_page == 'about' else '' }}">About Us</a>
      </li>
      <li>
        <a href="{{ url_for('contact') }}"
           class="{{ 'active' if active_page == 'contact' else '' }}">Contact Us</a>
      </li>
    </ul>
  </nav>
  <!-- TASK 8.2: Persistent notification box inherited by every child view. -->
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% if messages %}
      <div class="notification-wrapper">
        {% for category, message in messages %}
          <div class="notification-box notification-{{ category }}">
            <span class="notification-text">{{ message }}</span>
            <span class="notification-close"
                  onclick="this.parentElement.style.display='none';">&times;</span>
          </div>
        {% endfor %}
      </div>
    {% endif %}
  {% endwith %}
  <main class="page-content">
    {% block content %}<!-- Child views override this block -->{% endblock %}
  </main>
  <footer class="site-footer">
    <p>&copy; 2026 MySite &mdash; Lab Sheet 08, Saiyed Shizain</p>
  </footer>
</body>
</html>
```

```
$ curl -s http://127.0.0.1:5000/ → 200 ('active' marker present on Home link)
```

```
$ curl -s http://127.0.0.1:5000/about → 200
$ curl -s http://127.0.0.1:5000/contact → 200
```

Conclusion. All three pages render with correct active states. — Saiyed Shizain

---

Task 8.2 — Persistent notification box via inherited hooks

Aim. One flash-message block in base.html inherited by every child view.

Theory. get_flashed_messages() drains the session queue; placement in the master layout makes it persistent across pages.

```
{% extends "base.html" %}
{% block title %}Home{% endblock %}
{% block content %}
  <h1>Welcome Home</h1>
  <p>
    This page inherits its navbar, active-link highlighting, and
    notification box entirely from <code>base.html</code>. Only this
    content block is unique to the Home view.
  </p>
{% endblock %}
```

```
{% extends "base.html" %}
{% block title %}About Us{% endblock %}
{% block content %}
  <h1>About Us</h1>
  <p>
    We are a small team demonstrating Jinja2 template inheritance:
    one master layout (<code>base.html</code>) and three specialized
    child views (Home, About Us, Contact Us) that each override only
    the content section.
  </p>
{% endblock %}
```

```
* { box-sizing: border-box; font-family: Arial, Helvetica, sans-serif; }
body { margin: 0; background: #f4f6f8; color: #222; }
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background: #1f2937;
  padding: 14px 32px;
}
.brand { color: #fff; font-weight: bold; font-size: 1.2rem; }
.nav-links { list-style: none; display: flex; gap: 24px; margin: 0; padding: 0; }
.nav-links a {
  color: #cbd5e1;
  text-decoration: none;
  padding: 6px 10px;
  border-radius: 4px;
  transition: background 0.2s, color 0.2s;
}
.nav-links a:hover { color: #fff; }
/* TASK 8.1: active nav utility highlight class, injected server-side */
.nav-links a.active {
  background: #2563eb;
  color: #fff;
  font-weight: bold;
}
/* TASK 8.2: persistent notification / feedback box */
.notification-wrapper { padding: 12px 32px 0; }
.notification-box {
  display: flex;
  justify-content: space-between;
```

```
  align-items: center;
  padding: 12px 16px;
  border-radius: 6px;
  margin-bottom: 10px;
  font-size: 0.95rem;
}
.notification-success { background: #d1fae5; color: #065f46; border: 1px solid #10b981;
}
.notification-error { background: #fee2e2; color: #991b1b; border: 1px solid #ef4444; }
.notification-close { cursor: pointer; font-weight: bold; margin-left: 12px; }
.page-content { padding: 32px; max-width: 720px; margin: 0 auto; }
.contact-form { display: flex; flex-direction: column; max-width: 480px; }
.contact-form label { margin-top: 12px; font-weight: bold; }
.contact-form input, .contact-form textarea {
  padding: 8px; margin-top: 4px; border: 1px solid #ccc; border-radius: 4px;
}
.contact-form button {
  margin-top: 18px; padding: 10px; background: #2563eb; color: #fff;
  border: none; border-radius: 4px; cursor: pointer; font-weight: bold;
}
.contact-form button:hover { background: #1d4ed8; }
.site-footer { text-align: center; padding: 16px; color: #6b7280; font-size: 0.85rem; }
```

Conclusion. Children override only content; notifications come free. — Saiyed Shizain

---

Task 8.3 — Contact form with server-log verification

Aim. Validate name/email/message, log verified feedback, flash confirmation.

Theory. Empty fields → 400; malformed email → 400; only verified text reaches the dedicated feedback_logger (file + console).

```
"""
Lab Sheet 08: Template Inheritance Blueprints & Modular UI Engineering
-----------------------------------------------------------------------
Flask application demonstrating:
 8.1 - programmatic active-nav-link markers via context variables
 8.2 - inherited, persistent flash-message notification box
 8.3 - a Contact Us form that validates input and forwards verified
       feedback text straight into the server logs
By Saiyed Shizain.
"""
import logging
from flask import Flask, render_template, request, redirect, url_for, flash

app = Flask(__name__)
app.secret_key = "lab08-secret-key"  # required for session-based flash messages

# --- Server log configuration ------------------------------------------
logger = logging.getLogger("feedback_logger")
logger.setLevel(logging.INFO)
handler = logging.FileHandler("feedback.log")
handler.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s | %(message)s"))
logger.addHandler(handler)
# also echo to console so it is visible when running the dev server
console = logging.StreamHandler()
console.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s | %(message)s"))
logger.addHandler(console)


@app.route("/")
def home():
    # `active_page` tells base.html which nav link gets the .active class
    return render_template("home.html", active_page="home")
```

```
@app.route("/about")
def about():
    return render_template("about.html", active_page="about")


@app.route("/contact", methods=["GET", "POST"])
def contact():
    if request.method == "POST":
        name = request.form.get("name", "").strip()
        email = request.form.get("email", "").strip()
        message = request.form.get("message", "").strip()
        # --- Task 8.3: verification / validation before logging ---------
        if not name or not email or not message:
            flash("Please fill in every field before submitting.", "error")
            return render_template("contact.html", active_page="contact"), 400
        if "@" not in email or "." not in email:
            flash("Please enter a valid email address.", "error")
            return render_template("contact.html", active_page="contact"), 400
        # Verified: forward the feedback text into the server logs
        logger.info("New feedback received | name=%s | email=%s | message=%s",
                    name, email, message)
        flash(f"Thanks {name}! Your feedback has been received.", "success")
        return redirect(url_for("contact"))
    return render_template("contact.html", active_page="contact")


... [3 lines truncated for print]
```

```
{% extends "base.html" %}
{% block title %}Contact Us{% endblock %}
{% block content %}
  <h1>Contact Us</h1>
  <p>Send us your feedback below. Verified submissions are written to the server log.</p
>
  <form class="contact-form" method="POST" action="{{ url_for('contact') }}">
    <label for="name">Name</label>
    <input type="text" id="name" name="name" required
           value="{{ request.form.get('name', '') }}">
    <label for="email">Email</label>
    <input type="email" id="email" name="email" required
           value="{{ request.form.get('email', '') }}">
    <label for="message">Feedback message</label>
    <textarea id="message" name="message" rows="5" required>{{ request.form.get('message
', '') }}</textarea>
    <button type="submit">Submit Feedback</button>
  </form>
{% endblock %}
```

```
$ curl -X POST name/email/message → 302, follow-up shows:
Thanks Saiyed Shizain! Your feedback has been received.
$ curl -X POST empty fields → 400
$ curl -X POST bad email → 400
$ cat feedback.log
2026-10-15 00:42:48 | INFO | New feedback received | name=Saiyed Shizain |
email=saiyed@example.com | message=Great site!
```

Conclusion. Genuine log line with my details; both rejection paths return 400. — Saiyed Shizain